

SPRAWOZDANIE Z ZADANIA NR 3

Mateusz Nowak, Mateusz Wójcicki

Kraków, 14 grudnia 2025.

1 Opis ćwiczenia

2 Compress Matrix

2.1 Pseudokod algorytmu CompressMatrix

- Dane wejściowe: $t_{\min}, t_{\max}, s_{\min}, s_{\max}$ – zakres indeksów bloku macierzy, r – zadany rząd aproksymacji.
- Obliczamy przybliżony rozkład SVD dla danego bloku:

$$[U, D, V] = \text{truncatedSVD}(A(t_{\min} : t_{\max}, s_{\min} : s_{\max}), r + 1).$$

- Jeżeli blok składa się wyłącznie z zer:
 - tworzymy nowy węzeł o randze 0,
 - zapisujemy rozmiar bloku,
 - zwracamy utworzony węzeł.
- W przeciwnym wypadku:
 - wyznaczamy rzeczywisty rząd aproksymacji jako $\text{rank} = r$,
 - tworzymy nowy węzeł o randze rank ,
 - zapisujemy pierwsze rank wartości osobliwe,
 - zapisujemy macierz $U(:, 1 : \text{rank})$,
 - zapisujemy iloczyn $D(1 : \text{rank}, 1 : \text{rank}) \cdot V(1 : \text{rank}, :)$,
 - zapisujemy rozmiar bloku.
- Zwracamy węzeł zawierający skompresowaną reprezentację bloku macierzy.

2.2 Fragmenty kodu

```
def CompressMatrix(M, t_min, t_max, s_min, s_max, r):
    [U, sigma, Vh, block] = truncatedSVD(M, t_min, t_max, s_min, s_max, r + 1)

    if np.allclose(block, 0):
        v = Node(t_min, t_max, s_min, s_max)
        return v

    actual_rank = min(r, len(sigma))

    v = Node(t_min, t_max, s_min, s_max, rank=actual_rank)
    v.singularvalues = sigma[:actual_rank]
    v.U = U[:, :actual_rank]
    D = np.diag(sigma[:actual_rank])
    v.V = D @ Vh[:actual_rank, :]

    return v
```

Rysunek 1: Compress Matrix

3 Create Tree

3.1 Pseudokod algorytmu CreateTree

- Dane wejściowe: $t_{\min}, t_{\max}, s_{\min}, s_{\max}$ – zakresy indeksów wierszy i kolumn bloku macierzy A , r – maksymalny rząd aproksymacji, ε – próg dokładności.

- Obliczamy przybliżony rozkład SVD dla aktualnego bloku macierzy:

$$[U, D, V] = \text{truncatedSVD}(A(t_{\min} : t_{\max}, s_{\min} : s_{\max}), r + 1).$$

- Sprawdzamy warunek dopuszczalności bloku:
 - jeśli rozmiar bloku jest mniejszy lub równy r , blok uznajemy za dopuszczalny,
 - w przeciwnym wypadku sprawdzamy, czy $(r + 1)$ -ta wartość osobliwa spełnia warunek

$$D_{r+1, r+1} < \varepsilon.$$

- Jeżeli blok jest dopuszczalny:
 - wywołujemy procedurę **CompressMatrix** dla aktualnego zakresu indeksów,
 - zwracamy węzeł zawierający skompresowaną reprezentację bloku.
- Jeżeli blok nie jest dopuszczalny:
 - tworzymy nowy węzeł drzewa bez danych,
 - dzielimy blok na cztery podbloki (podział na połowy wierszy i kolumn),
 - rekurencyjnie wywołujemy **CreateTree** dla każdego z czterech podbloków:
 - * $(t_{\min}, t_{\text{new}}, s_{\min}, s_{\text{new}})$,
 - * $(t_{\min}, t_{\text{new}}, s_{\text{new}} + 1, s_{\max})$,
 - * $(t_{\text{new}} + 1, t_{\max}, s_{\min}, s_{\text{new}})$,
 - * $(t_{\text{new}} + 1, t_{\max}, s_{\text{new}} + 1, s_{\max})$,
 - dodajemy uzyskane węzły jako synów bieżącego węzła.
- Zwracamy utworzony węzeł drzewa.

3.2 Fragmenty kodu

```
def CreateTree(M, t_min, t_max, s_min, s_max, r, e):
    [U, sigma, Vh, block] = truncatedSVD(M, t_min, t_max, s_min, s_max, r + 1)

    current_size = min(block.shape)

    if current_size <= r:
        is_admissible = True
    else:
        if sigma[r] < e:
            is_admissible = True
        else:
            is_admissible = False

    if is_admissible:
        v = CompressMatrix(M, t_min, t_max, s_min, s_max, r)
    else:
        v = Node(t_min, t_max, s_min, s_max)

    t_new_max = t_min + (t_max - t_min) // 2
    s_new_max = s_min + (s_max - s_min) // 2

    v.sons.append(CreateTree(M, t_min, t_new_max, s_min, s_new_max, r, e))
    v.sons.append(CreateTree(M, t_min, t_new_max, s_new_max + 1, s_max, r, e))
    v.sons.append(CreateTree(M, t_new_max + 1, t_max, s_min, s_new_max, r, e))
    v.sons.append(CreateTree(M, t_new_max + 1, t_max, s_new_max + 1, s_max, r, e))

    return v
```

Rysunek 2: Create Tree

4 Wyniki kompresji obrazów

4.1 Obraz wejściowy

- Nieskompresowana bitmapa (obraz oryginalny)
- Składowe kolorów: R, G, B



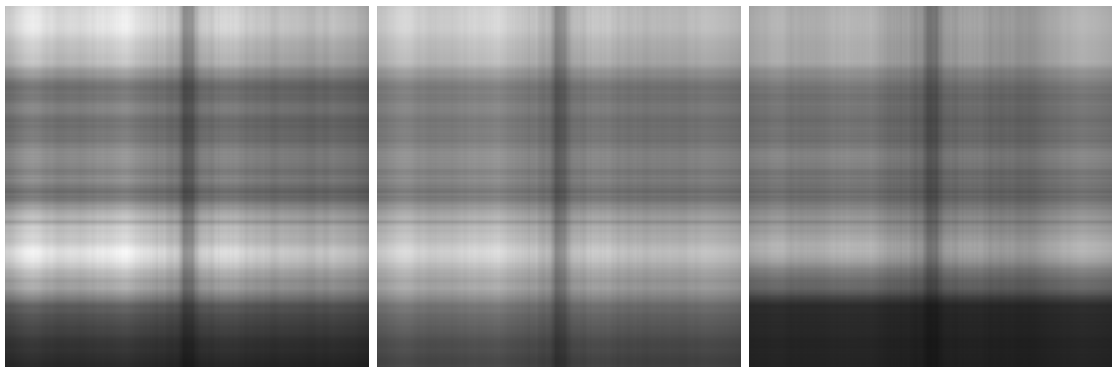
Rysunek 3: Nieskompresowana bitmapa RGB



Rysunek 4: Składowe kolorów R, G, B obrazu wejściowego

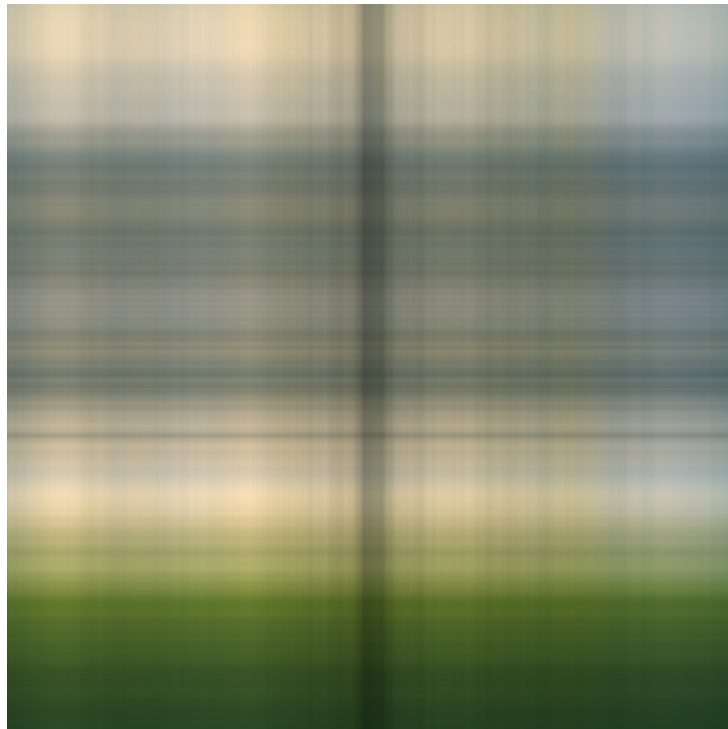
4.2 Metoda kompresji 1 (parametr $r = 1, \delta = \sigma_1$)

4.2.1 Wynikowe bitmapy składowych



Rysunek 5: Zrekonstruowane bitmapy R, G, B ($\delta = d_1$)

4.2.2 Obraz wynikowy RGB



Rysunek 6: Obraz RGB po kompresji ($\delta = d_1$)

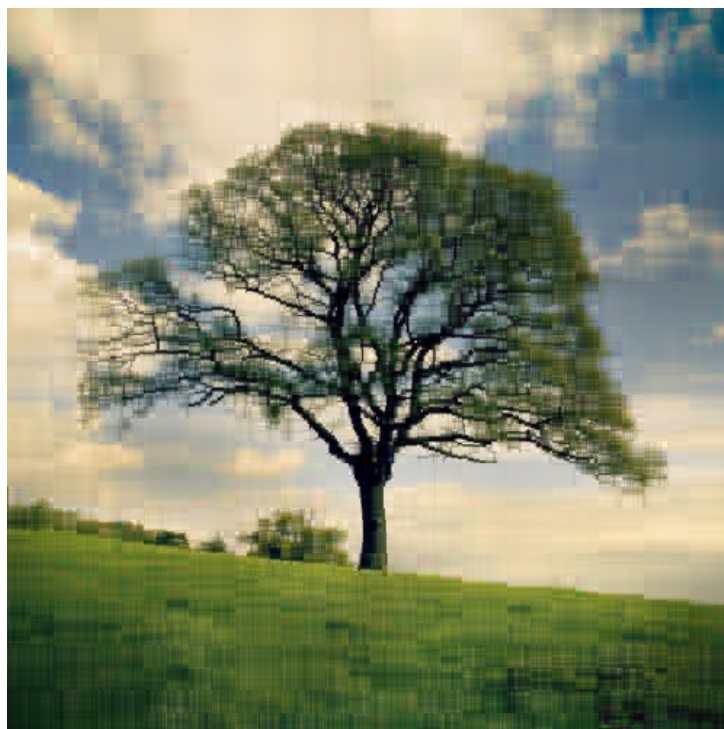
4.3 Metoda kompresji 1 (parametr $r = 1, \delta = \sigma_{2^k}$)

4.3.1 Wynikowe bitmapy składowych



Rysunek 7: Zrekonstruowane bitmapy R, G, B ($\delta = \sigma_{2^k}$)

4.3.2 Obraz wynikowy RGB



Rysunek 8: Obraz RGB po kompresji ($\delta = d_1$)

—

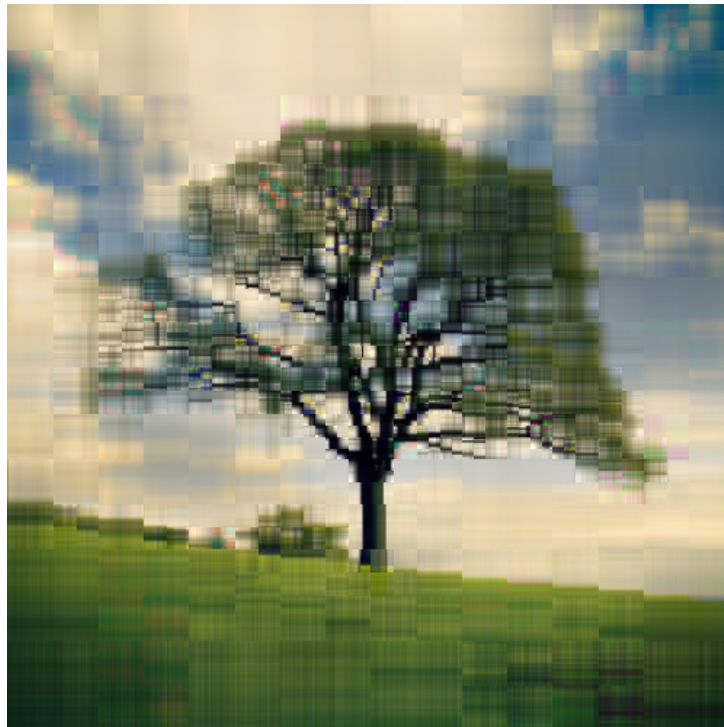
4.4 Metoda kompresji 1 (parametr $r = 1, \delta = \sigma_{2^k/2}$)

4.4.1 Wynikowe bitmapy składowych



Rysunek 9: Zrekonstruowane bitmapy R, G, B ($\sigma_{2^k/2}$)

4.4.2 Obraz wynikowy RGB

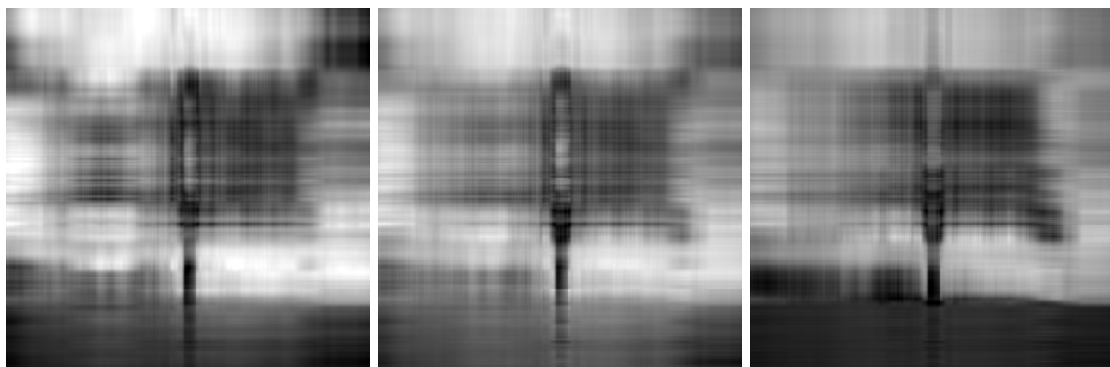


Rysunek 10: Obraz RGB po kompresji ($\delta = d_1$)

—

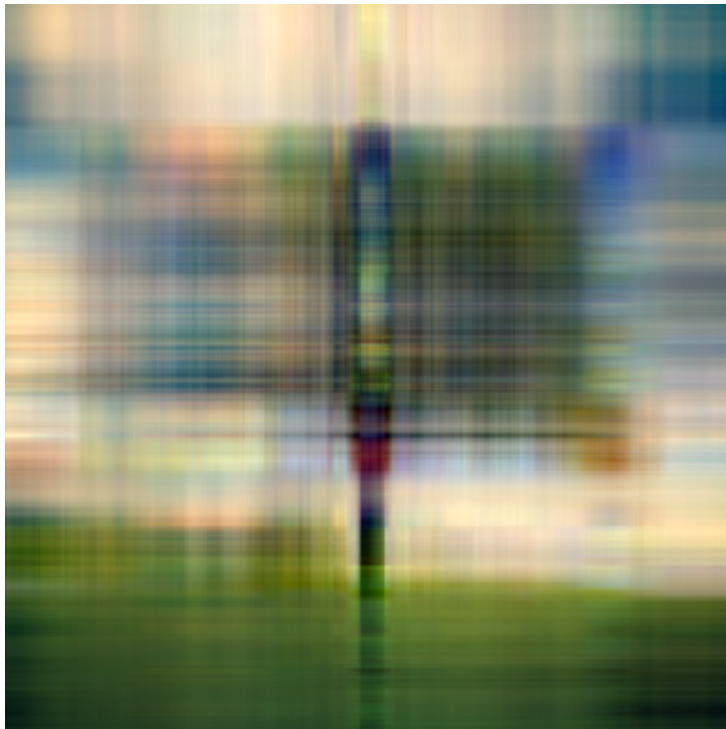
4.5 Metoda kompresji 1 (parametr $r = 4, \delta = \sigma_1$)

4.5.1 Wynikowe bitmapy składowych



Rysunek 11: Zrekonstruowane bitmapy R, G, B ($\delta = d_1$)

4.5.2 Obraz wynikowy RGB



Rysunek 12: Obraz RGB po kompresji ($\delta = d_1$)

4.6 Metoda kompresji 1 (parametr $r = 4, \delta = \sigma_{2^k}$)

4.6.1 Wynikowe bitmapy składowych



Rysunek 13: Zrekonstruowane bitmapy R, G, B ($\delta = d_1$)

4.6.2 Obraz wynikowy RGB



Rysunek 14: Obraz RGB po kompresji ($\delta = d_1$)

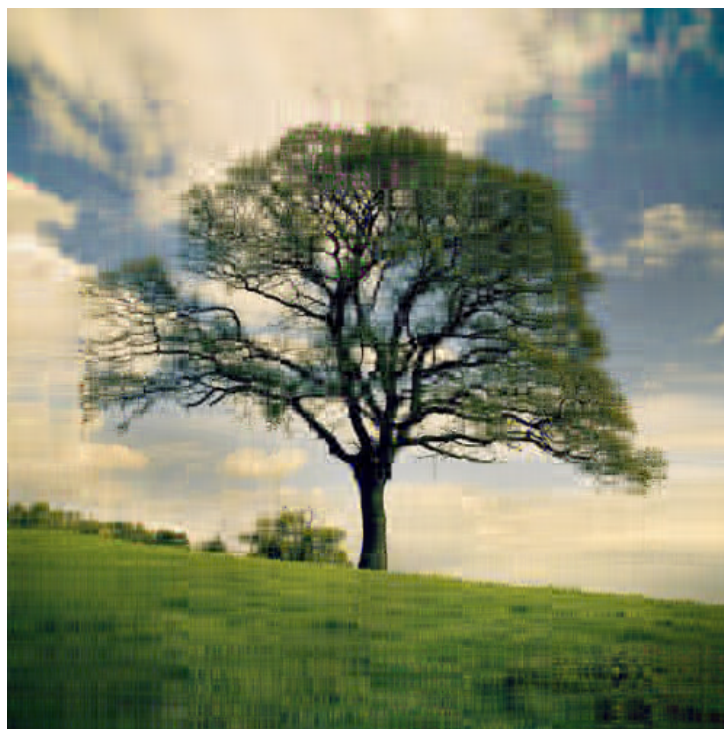
4.7 Metoda kompresji 1 (parametr $r = 4, \delta = \sigma_{2^k/2}$)

4.7.1 Wynikowe bitmapy składowych



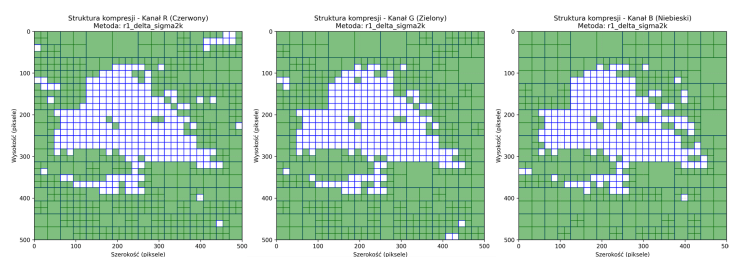
Rysunek 15: Zrekonstruowane bitmapy R, G, B ($\delta = d_1$)

4.7.2 Obraz wynikowy RGB



Rysunek 16: Obraz RGB po kompresji ($\delta = d_1$)

4.7.3 Macierze kompresji



Rysunek 17: Macierz)